



ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA DE TELECOMUNICACIÓN

Fundamentos de Inteligencia Artificial

Técnicas básicas de optimización

José A. Montenegro

5 de febrero de 2026



Contenido

Búsqueda Local

Hill Climbing

Recorrido Simulado

Algoritmos Genéticos



Búsqueda local



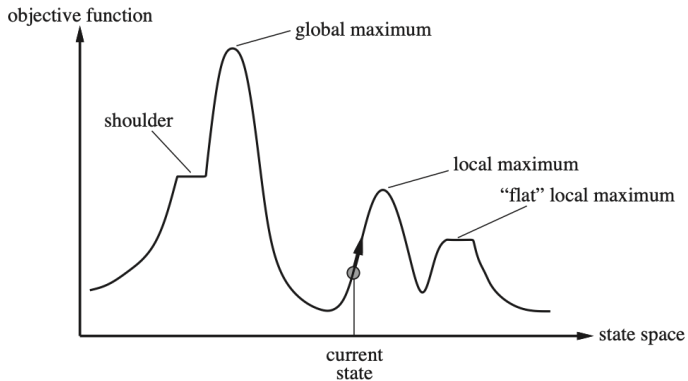


Búsqueda Local

- Los algoritmos anteriores exploran los espacios de búsqueda y la solución a los problemas son el camino desde el origen al objetivo.
 - Existen problemas donde el camino al objetivo es irrelevante.
- Búsqueda local son algoritmos útiles para resolver problemas de optimización, encontrar el mejor estado según función objetivo.
- El espacio de búsqueda está formado por estados, que vienen dados por el valor de una función objetivo, que puede ser maximizar o minimizar.



Búsqueda Local





Búsqueda local Hill Climbing





Hill Climbing

- El algoritmo se mueve continuamente en dirección del valor creciente (maximizar), es decir, cuesta arriba.
- Finaliza cuando alcanza “un pico” donde ningún vecino tiene un valor más alto (maximizar).
- El algoritmo no mantiene un árbol de búsqueda, solo nodo actual y valor función objetivo.
- Similar a Greedy en que utiliza sólo la heurística, pero no busca eun camino distinto, ya que no tiene memoria de donde ha estado.



Hill Climbing

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  current  $\leftarrow$  problem.INITIAL
  while true do
    neighbor  $\leftarrow$  a highest-valued successor state of current
    if VALUE(neighbor)  $\leq$  VALUE(current) then return current
    current  $\leftarrow$  neighbor
```

Figure 4.2 The hill-climbing search algorithm, which is the most basic local search technique. At each step the current node is replaced by the best neighbor.

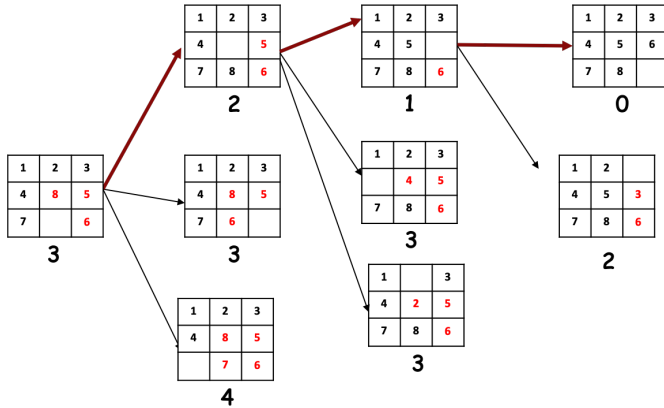


Hill Climbing (Minimizar) en Python

```
def hill_climbing(problem, max_iterations=1000):  
    current_state = problem.state  
    current_cost = problem.calculate_cost(current_state)  
    print ("current_cost:", current_cost)  
  
    for _ in range(max_iterations):  
        neighbors = problem.get_neighbors()  
        print ("\t neighbors:", neighbors)  
  
        best_neighbor      = min(neighbors, key=lambda state: problem.calculate_cost(state))  
        best_neighbor_cost = problem.calculate_cost(best_neighbor)  
        print ("\t\t best_neighbor:", best_neighbor, " - ", best_neighbor_cost)  
  
        if best_neighbor_cost >= current_cost:  
            return current_state  
  
    problem.state = best_neighbor  
    current_state = best_neighbor  
    current_cost = best_neighbor_cost
```



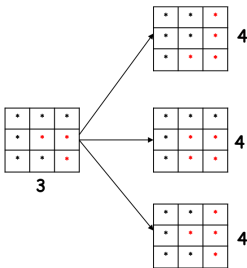
Hill Climbing - Puzzle 8





Hill Climbing - Problemas Hill Climbing

- **Máximo Local:** El valor no es el punto más elevado en el espacio de estados.
- **Meseta:** El espacio tiene una región plana, que si coincide con un máximo local no hay solución posible.
- **Remedio:** Comienzos estado aleatorio.





Ejemplo 8 Reinas

- El objetivo del problema es establecer 8 reinas en un tablero de forma que no se ataquen entre sí.
- Evitar Misma fila, columna o diagonal.

Estado : Cualquier representación de 8 reinas en el tablero.

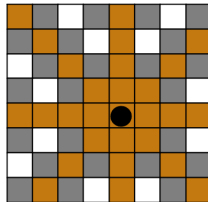
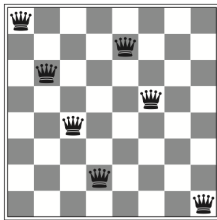
Estado Inicial : Tablero con 8 reinas

Acciones : Mover una reina a una casilla vacía de una columna

Modelo Transición : Devuelve tablero con una reina añadida en una casilla determinada.

Test Objetivo : 8 reinas en el tablero que no se ataquen entre sí.

Función Heurística : Número de pares de reinas que se atacan (MIN)



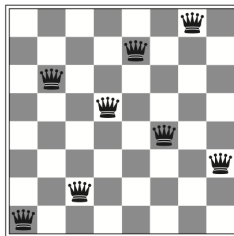


Ejemplo 8 Reinas

- Los sucesores de un estado son todos los posibles estados generados al mover una reina en una columna ($8 \times 7 = 56$ sucesores).
- Partiendo de (1) llegamos a (2) en 5 pasos y queda en mínimo local.

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♙	13	16	13	16
♙	14	17	15	♙	14	16	16
17	♙	16	18	15	♙	15	♙
18	14	♙	15	15	14	♙	16
14	14	13	17	12	14	12	18

(1) $h = 3 + 4 + 2 + 3 + 2 + 2 + 1 = 17$



(2) $h = 0 + 0 + 0 + 1 + 0 + 0 + 0 + 0 = 1$



Búsqueda local Recorrido Simulado





Recorrido Simulado

- Algoritmo anterior no realiza movimientos “cuesta abajo”, no es completo ya que puede estancarse en un máximo local.
- Un algoritmo aleatorio, es completo, pero puramente ineficaz.
- Combinamos las dos visiones que permita un algoritmo eficaz y completo.
- Comienza a una temperatura alta y luego reduciendo gradualmente la intensidad, a una temperatura más baja.
- Algoritmo parecido anterior, en vez de escoger el mejor movimiento, escoge un **movimiento aleatorio**.



Recorrido Simulado

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

current $\leftarrow$ *problem*.INITIAL

for $t = 1$ **to** ∞ **do**

$T \leftarrow$ *schedule*(t)

if $T = 0$ **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

$\Delta E \leftarrow$ VALUE(*current*) – VALUE(*next*)

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $e^{-\Delta E/T}$

Figure 4.4 The simulated annealing algorithm, a version of stochastic hill climbing where some downhill moves are allowed. The *schedule* input determines the value of the “temperature” T as a function of time.



Recorrido Simulado en Python 1/2

```
def simulated_annealing(problem, max_iterations=10000000):  
    current_state = problem.state  
    current_cost = problem.calculate_cost(current_state)  
    debug(f"current_state, {current_state}: {current_cost}")  
  
    current_temperature = max_iterations  
  
    while (not problem.is_goal_state()) and (current_temperature > 0):  
        neighbors = problem.get_neighbors()  
        debug(f"\t Neighbors:, {neighbors}")  
  
        next_state = random.choice(neighbors)  
        next_cost = problem.calculate_cost(next_state) # Calculando el costo del próximo  
        debug(f"\t\t next_state {next_state} ({next_cost})")
```



Recorrido Simulado en Python 2/2

```
diff_cost = next_cost - current_cost
move = False
if (diff_cost <= 0):
    move = True
else:
    boltz = math.exp(-float(diff_cost) / current_temperature)
    randomvalue = random.random()
    debug(f"\t\t diff_cost {diff_cost}")
    debug(f"\t\t randomvalue {randomvalue}, boltz {boltz}")
    if randomvalue <= boltz:
        move = True
if move:
    problem.state = next_state
    current_state = next_state
    current_cost = next_cost
    debug(f"current_state, {problem.state}: {current_cost}")

current_temperature-=1

if (problem.is_goal_state()):
    print (f"Solución encontrada en: {current_temperature}/{max_iterations}")

return current_state
```



Puzzle 8: Energía, Recorrido Simulado

1	2	3
4	5	6
7	8	9

Objetivo

[1,2,3,4,5,6,7,8,0]
 $|1-1|+|2-2|+|3-3|+|4-4|+|5-5|+|6-6|+|7-7|+|8-8|$
 0 Energía

4	1	7
6		2
3	5	8

[4,1,7,6,0,2,3,5,8]
 $|1-2|+|2-6|+|3-7|+|4-1|+|5-8|+|6-4|+|7-3|+|8-9|$
 $1+4+4+3+3+2+4+1=22$ Energía

arriba

izq

dch

abajo

4		7
6	1	2
3	5	8

[4,0,7,6,1,2,3,5,8]
 $|1-5|+|2-6|+|3-7|+|4-1|+|5-8|+|6-4|+|7-3|+|8-9|$
 $4+4+4+3+3+2+4+1=25$ Energía

4	1	7
	6	2
3	5	8

[4,1,7,0,6,2,3,5,8]
 $|1-2|+|2-6|+|3-7|+|4-1|+|5-8|+|6-5|+|7-3|+|8-9|$
 $1+4+4+3+3+1+4+1=21$ Energía

4	1	7
6	2	
3	5	8

[4,1,7,6,2,0,3,5,8]
 $|1-2|+|2-5|+|3-7|+|4-1|+|5-8|+|6-4|+|7-3|+|8-9|$
 $1+3+4+3+3+2+4+1=21$ Energía

4	1	7
6	5	2
3		8

[4,1,7,6,5,2,3,0,8]
 $|1-2|+|2-6|+|3-7|+|4-1|+|5-5|+|6-4|+|7-3|+|8-9|$
 $1+4+4+3+0+2+4+1=19$ Energía



Recorrido Simulado en Python 2/2

```
diff_cost = next_cost - current_cost
move = False
if (diff_cost <= 0):
    move = True
else:
    boltz = math.exp(-float(diff_cost) / current_temperature)
    randomvalue = random.random()
    debug(f"\t\t diff_cost {diff_cost}")
    debug(f"\t\t randomvalue {randomvalue}, boltz {boltz}")
    if randomvalue <= boltz:
        move = True
```

diff_cost = next_cost - current_cost

= 19 - 22 = -3

= 21 - 22 = -1

= 25 - 22 = 3

boltz = math.exp(-float(diff_cost) / current_temperature)

current_temperature= 40 -> $e^{-3/40} = 0.9277 = 92.77 \%$



Búsqueda local Algoritmos Genéticos





Esquema General





Esquema General

INICIALIZA población con soluciones aleatorias
EVALUA cada candidato

REPEAT UNTIL (Condición finalización) DO

SELECCIONA padres;
CRUZA pares de padres;
MUTA la descendencia;
EVALUA los nuevos candidatos;
SELECCIONA individuos para la próxima

generación

OD



Definiciones

Población : Comenzamos con un conjunto de k estados generados de forma aleatoria.

Función idoneidad : Cada estado se tasa con una función de evaluación. Valores más altos para los mejores valores.

Cruce : Seleccionamos dos pares de forma aleatoria para la reproducción, según con las probabilidades. Además se selecciona aleatoriamente un punto de cruce.

Mutación : Finalmente se realiza una mutación aleatoria con una probabilidad independiente.



Algoritmo pseudocódigo

```
function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
    weights  $\leftarrow$  WEIGHTED-BY(population, fitness)
    population2  $\leftarrow$  empty list
    for i = 1 to SIZE(population) do
      parent1, parent2  $\leftarrow$  WEIGHTED-RANDOM-CHOICES(population, weights, 2)
      child  $\leftarrow$  REPRODUCE(parent1, parent2)
      if (small random probability) then child  $\leftarrow$  MUTATE(child)
      add child to population2
    population  $\leftarrow$  population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness

function REPRODUCE(parent1, parent2) returns an individual
  n  $\leftarrow$  LENGTH(parent1)
  c  $\leftarrow$  random number from 1 to n
  return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))
```

Figure 4.7 A genetic algorithm. Within the function, *population* is an ordered list of individuals, *weights* is a list of corresponding fitness values for each individual, and *fitness* is a function to compute these values.



Ejemplo 8 Reinas - Genéticos

- El objetivo del problema es establecer 8 reinas en un tablero de forma que no se ataquen entre sí.
- Evitar Misma fila, columna o diagonal.

Estado : Cualquier representación de 8 reinas en el tablero.

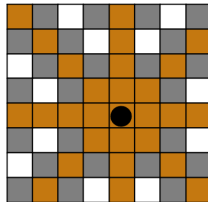
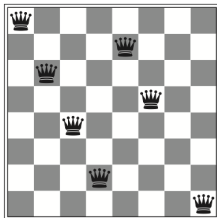
Estado Inicial : Tablero con 8 reinas

Acciones : Mover una reina a una casilla vacía de una columna

Modelo Transición : Devuelve tablero con una reina añadida en una casilla determinada.

Test Objetivo : 8 reinas en el tablero que no se ataquen entre sí.

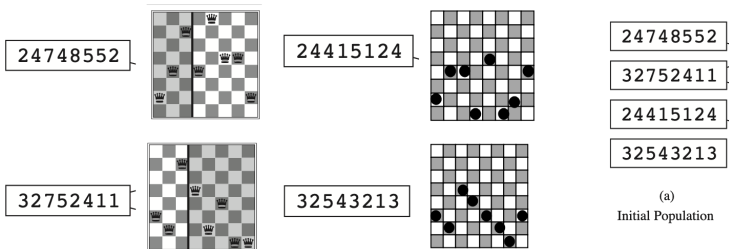
Función Heurística : Número de pares de reinas que se atacan (MIN)





Ejemplo 8 Reinas - Genéticos

Representación Problema

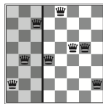




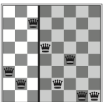
Ejemplo 8 Reinas - Genéticos

Función Evaluación: Número de pares de reinas que “no atacan”

24748552



32752411



	1	2	3	4	5	6	7	8
1		1	2	3	4	5	6	SI
2			7	SI	8	9	10	11
3				12	13	14	15	SI
4					16	17	18	19
5						20	21	22
6							SI	23
7								24
8								

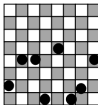
	1	2	3	4	5	6	7	8
1		SI	1	2	3	4	5	6
2			7	8	SI	9	10	11
3				12	13	SI	14	15
4					16	17	18	SI
5						19	20	21
6							22	23
7								SI
8								



Ejemplo 8 Reinas - Genéticos

Función Evaluación

24415124



32543213



	1	2	3	4	5	6	7	8
1								
2								
3								
4								
5								
6								
7								
8								

	1	2	3	4	5	6	7	8
1								
2								
3								
4								
5								
6								
7								
8								

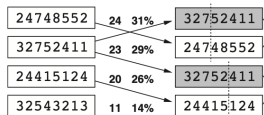


Ejemplo 8 Reinas - Genéticos

Evaluación y Selección

	Función Evaluación	Selección	
24748552	24	31%	$24/(24+23+20+11)=$ $24/78=0,307$
32752411	23	29%	$23/78=0,29$
24415124	20	26%	$20/78=0,256$
32543213	11	14%	$11/78=0,14$

(a)
Initial Population

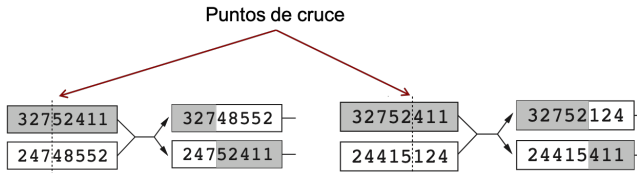




Ejemplo 8 Reinas - Genéticos

Cruce

1. Selecciona un punto de cruce aleatorio
2. Copia la primera parte de un padre al hijo
3. Copia la segunda parte de otro padre al hijo





Ejemplo 8 Reinas - Genéticos

Mutación

- Posibles Operaciones:
 - Intercambia dos valores de posición
 - Cambia el valor aleatorio de una posición
- Puede resultar una configuración no válida

32748¹52

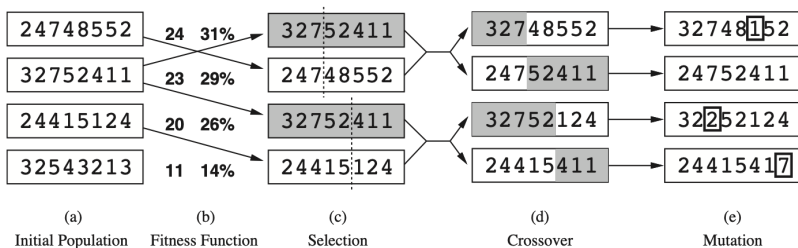
2441541⁷

32²52124

24752411



Ejemplo 8 Reinas - Genéticos





That's all folks!

